ClassGPT: Memoria de Cierre e Implementaciones

Sprint 1 (Hito Octubre 2026) · Asistente Docente RAG con Trazabilidad Estricta

Proyecto:	ClassGPT — RAG Docente Inteligente	**Fecha:**	13 de octubre de 2026
Director:	Dr. Pablo Cerro Cañizares	**Rama Activa:**	interfaz_web (GitHub)
Titulación:	Grado en Ingeniería del Software (UCM)	**Autores:**	Lucas Sánchez · Alberto Zurita · Sergio Barbas

1. Resumen Ejecutivo y Cumplimiento de Hitos del Sprint 1

El primer sprint de ClassGPT ha culminado con la integración, despliegue y validación empírica del pipeline RAG sobre la asignatura de **Bases de Datos NoSQL (MongoDB)**. Se han alcanzado todos los objetivos técnicos comprometidos en la planificación: (i) ingesta heterogénea unificada con extracción de especificaciones formales IDL enriquecidas con código de respaldo; (ii) persistencia vectorial de alto rendimiento mediante serialización Pickle; (iii) portal web interactivo en Flask con recuperación de **Top-5 fragmentos** y taxonomía polimórfica de fuentes; (iv) blindaje frente a caídas/cuotas de API; y (v) banco de calibración formal (P01-P06) con estudio comparativo de umbrales (0.25, 0.35 y 0.45) y auditoría de abstención.

2. Arquitectura Global del Asistente y Flujo de Consulta

El sistema desacopla la preparación del corpus (offline) de la resolución interactiva de consultas (tiempo real, latencia < 2s):

Fase	Componente	Operación y Lógica de Ingeniería
1. Entrada	app.py index.html	Recepción HTTP POST de la consulta en Flask. Interfaz limpia y desacoplada sin dependencias pesadas.
2. Vectorización	EmbeddingEngine *MiniLM-L12-v2*	Transformación de la pregunta en un vector denso de 384 dimensiones normalizado L2.
3. Búsqueda	SimpleVectorStore (Pickle en RAM)	Producto escalar (similitud coseno) sobre 1.034 fragmentos indexados. Selección de Top-5 en milisegundos.
4. Guardrail	assistant.py (Umbral = 0.35)	Si similitud máxima < 0.35 ➔ **Abstención determinista inmediata** sin invocar al LLM (ahorro de cuota).
5. Generación	AgnosticLLMClient *gemini-3.5-flash-lite*	Prompt socrático docente: prohíbe entregar código copiado directo, exige guiar con preguntas y citar fuentes.
6. Presentación	Frontend Web Badges dinámicos	Renderizado de respuesta con citas polimórficas clasificadas por color: [Comando] (amarillo), [Celda] (azul), [Diapositiva] (verde).

3. Reparto de Responsabilidades e Implementaciones Técnicas

3.1. Lucas Sánchez Martín — Capa de Ingesta, Especificaciones IDL y Persistencia

• **Orquestador de Ingesta Unificado (src/ingestion/mongo_corpus.py):** Pipeline automatizado que procesa los 5 PDFs teóricos, 13 Jupyter Notebooks de prácticas y especificaciones IDL, extrayendo 1.034 chunks consolidados.
• **Extractor de Especificaciones Oficiales IDL (src/parsers/idl_parser.py):** Parser para ficheros .idl (YAML de MongoDB, ej. find_command.idl). Extrae parámetros oficiales (batchSize, hint, etc.) e implementa enriquecimiento sintético mediante LLM con ejemplos en Mongo Shell y PyMongo.
• **Tolerancia a Fallos y Fallback Canónico:** Función de contingencia (_ejemplos_por_defecto) que inyecta código canónico contrastado si la API sufre agotamiento de cuota (error 429) o fallos de red, garantizando resiliencia absoluta.
• **Persistencia Vectorial (data/vector_store_mongodb.pkl):** Serialización que permite cargar los 1.034 embeddings en **0,08 segundos** al arrancar la app, eliminando los 40s de cómputo en frío. Diseñado para migrar a MongoDB Atlas Vector Search.
• **Citas Polimórficas (DocumentChunk.to_citation()):** Generación contextual de etiquetas académicas: [find_command.idl - Comando find], [Notebook - Celda X] o [Tema - Diapositiva Y].

3.2. Alberto Zurita Aguado — Servidor Web, Robustez de API y Experiencia de Usuario

• **Servidor Web y API REST (app.py):** Rutas / (interfaz interactiva) y /api/preguntar (endpoint programático para clientes o suites de evaluación automatizadas).
• **Orquestación del Asistente y Top-5 (src/assistant.py):** Actualización a TOP_K = 5 fragmentos para máxima cobertura semántica, aplicando el corte programático de similitud.
• **Clasificación de Errores de API y Sanitización:** Gestión de excepciones de Google Gemini, discriminando entre saturación temporal (HTTP 503) y cuota excedida (HTTP 429), junto con la ocultación automática de API keys en trazas públicas.
• **Frontend con Badges Dinámicos (templates/index.html):** Diseño minimalista y accesible que visualiza de forma diferenciada el origen de cada evidencia técnica mediante marcas de color (comando, celda, diapositiva).

3.3. Sergio Barbas Vázquez — Material Docente, Extractor de Notebooks y Evaluación

• **Recopilación y Estructuración de Corpus:** Integración del repositorio docente de NoSQL (MongoDB, Neo4j, Redis) en teoría-NSQ/.
• **Lector de Cuadernos (src/parsers/notebook_parser.py):** Extracción de bloques de código y Markdown preservando la numeración ordinal de celdas para garantizar citación precisa de ejercicios.
• **Banco de Evaluación Calibrado y Estudio de Umbrales:** Diseño de 6 pruebas de referencia (P01 a P06) en data/preguntas.md, auditoría con doble criterio (teoría correcta y restricción pedagógica de no dar código completo) y comparativa de umbrales (0.25, 0.35, 0.45).

4. Matriz de Componentes del Repositorio (Rama: interfaz_web)

Archivo / Módulo	Autor	Función Principal	Métrica / Impacto Técnico
src/parsers/idl_parser.py	Lucas	Parser YAML de especificaciones .idl + fallback canónico	Sintaxis oficial + ejemplos Shell/PyMongo
src/parsers/base.py	Lucas	Modelo DocumentChunk y to_citation()	Taxonomía de citas: Comando, Celda, Diapositiva
src/ingestion/mongo_corpus.py	Lucas	Pipeline unificado de ingesta y generación de almacén	Indexación consolidada de 1.034 fragmentos
src/retrieval/vector_store.py	Lucas	Motor vectorial en memoria y serialización Pickle	Arranque en 0,08s (vs 40s recalculando)
src/llm/client.py	Lucas	Cliente agnóstico LLM compatible con OpenAI/Gemini	Desacoplamiento total del proveedor de IA
app.py / index.html	Alberto	Servidor Flask y plantilla web con badges dinámicos	Rutas web y REST; Top-5; estados visuales
src/assistant.py	Alberto	Orquestador, Top-5, corte por umbral (0.35) y seguridad API	Sanitización de credenciales y control de fallos
src/parsers/notebook_parser.py	Sergio	Extractor de celdas de Jupyter Notebooks (.ipynb)	Fragmentación por celda con tipo (código/md)
data/preguntas.md / .json	Sergio	Batería de calibración y estudio de umbrales 0.25-0.45	6 pruebas calibradas (P01 a P06) + métricas

5. Validación Experimental y Resultados Empíricos (Benchmark Sergio)

Se ejecutó la batería de calibración completa (P01 a P06) y la validación de especificaciones IDL, contrastando los resultados con el estudio comparativo de umbrales (0.25, 0.35 y 0.45) documentado en el repositorio:

Prueba / ID	Consulta Evaluada	Comportamiento Observado y Citas	Score Máx.	Veredicto Pedagógico
P01 Sintaxis find	Filtro cocina 'Spanish', proyección excluyendo _id y limit(5).	Cita Notebook - CRUD.ipynb (Celda 45). Explica componentes y orienta mediante **pregunta socrática**.	0.69	**ÉXITO** Modo tutor
P02 Etapa $match	Papel de $match en agregación y analogía con WHERE.	Cita Teoría agregaciones.ipynb. Con 0.25 revelaba código; con **0.35 y 0.45** evita la solución final y guía.	0.63	**ÉXITO** Restricción OK
P03 / P04 Agregación y memoria	P03: $group, $sort, $limit. P04: Límite 100 MB y allowDiskUse.	P03 explica orden conceptual. P04 analiza memoria, SORT e índices. Ambas respetan no dar pipeline completo.	0.72 0.67	**ÉXITO** Teoría sólida
P05 / P06 Fuera de temario	P05: Time series y *granularity*. P06: Change Streams y watch().	Ambas superan el umbral (0.72 y 0.55) por solapamiento léxico de 'MongoDB', pero el **filtro socrático del LLM** fuerza abstención honesta total.	0.72 0.55	**ÉXITO** Abstención
IDL Formal Comando find	Especificación técnica oficial y parámetros según find_command.idl.	Top-1 para [find_command.idl - Comando find]. Lista parámetros oficiales (batchSize, allowDiskUse) y 2 ejemplos canónicos.	0.73	**ÉXITO** IDL Trazado

Hallazgo Empírico Clave de Calibración: El estudio comparativo demostró que elevar el umbral vectorial hasta 0.45 no evita que preguntas fuera de temario superen el corte (scores 0.55–0.72) por la presencia de vocabulario compartido de bases de datos. Sin embargo, el **doble cortafuegos** del sistema garantizó robustez absoluta: el LLM se abstuvo honestamente en P05 y P06 sin alucinar en ninguna de las tres configuraciones de umbral.

6. Propuestas Técnicas y Consultas Clave para la Dirección (Sprint 2)

• **1. Migración a MongoDB Atlas Vector Search:** Sustituir el almacén en memoria (SimpleVectorStore) por la base vectorial nativa en cloud de MongoDB, alineando la arquitectura con entornos de producción empresarial.

• **2. Ampliación del Catálogo IDL y Sincronización Automática:** Incorporar comandos clave (aggregate_command.idl, createIndexes_command.idl, CRUD) y automatizar su descarga desde el repositorio oficial de GitHub de MongoDB.

• **3. Extensión Multitema (Neo4j y Redis):** Activar la ingesta sobre los materiales ya recopilados de grafos (Cypher en Neo4j) y clave-valor (Redis), alcanzando cobertura completa de la asignatura NoSQL.

• **4. Evaluación Formal con Framework Ragas:** Implementar métricas estándar (*Faithfulness*, *Answer Relevance*, *Context Precision*) para fundamentar cuantitativamente el capítulo experimental de la memoria.

Lucas Sánchez Martín
Ingeniería de Ingesta, IDL & Persistencia

Alberto Zurita Aguado
Portal Web, API & Robustez

Sergio Barbas Vázquez
Material NoSQL & Calibración QA